

Protección de Datos mediante Encriptación (AWS KMS y Encryption CLI)

 **Dificultad:** Intermedio

 **Tiempo Estimado:** 45 minutos

 **Servicios Principales:** AWS Key Management Service (KMS), Amazon EC2, AWS Systems Manager (Session Manager), AWS CLI.

Resumen y Objetivos

En este laboratorio, explorarás los fundamentos de la criptografía implementando encriptación simétrica para proteger información confidencial. Te conectarás a un servidor de archivos (Amazon EC2), crearás una clave criptográfica gestionada en AWS KMS y utilizarás la herramienta de línea de comandos **AWS Encryption CLI** para cifrar datos en texto plano y posteriormente descifrarlos, garantizando la privacidad e integridad de los archivos.

Objetivos alcanzados al finalizar:

-  Crear una clave de encriptación simétrica en AWS KMS.
 -  Instalar y configurar AWS Encryption CLI en una instancia Linux.
 -  Encriptar texto plano (*Plaintext*) transformándolo en texto cifrado (*Ciphertext*).
 -  Descifrar datos para recuperar la información original legible.
-

Análisis del Escenario (Diagnóstico Inicial)

La protección de datos en reposo es un pilar fundamental de la ciberseguridad corporativa. En este escenario, el cliente necesita un mecanismo robusto para asegurar archivos confidenciales alojados localmente en sus servidores.

Diagnóstico y Estrategia: En lugar de enviar los archivos a través de la red para ser cifrados por un servicio externo, implementarás una arquitectura de **Cifrado de Sobres (Envelope Encryption)** local. Proveerás al servidor con el *AWS Encryption CLI*, el cual se comunicará de forma segura con AWS KMS únicamente para obtener claves de datos (*Data Keys*). Estas claves cifrarán los archivos de forma ultra-rápida directamente en el almacenamiento local del servidor EC2.

Arquitectura del Laboratorio

1.  **AWS KMS:** Repositorio centralizado de alta seguridad (validado por FIPS 140-2) donde se creará y almacenará la Clave Maestra de Cliente (CMK) Simétrica.
2.  **Amazon EC2 (File Server):** Servidor Linux que aloja los archivos confidenciales. El acceso se realiza de forma segura mediante **SSM Session Manager** (sin abrir puertos SSH).
3.  **AWS Encryption CLI:** Herramienta de software instalada en el servidor que interactúa con KMS y ejecuta los algoritmos matemáticos de cifrado/descifrado localmente.

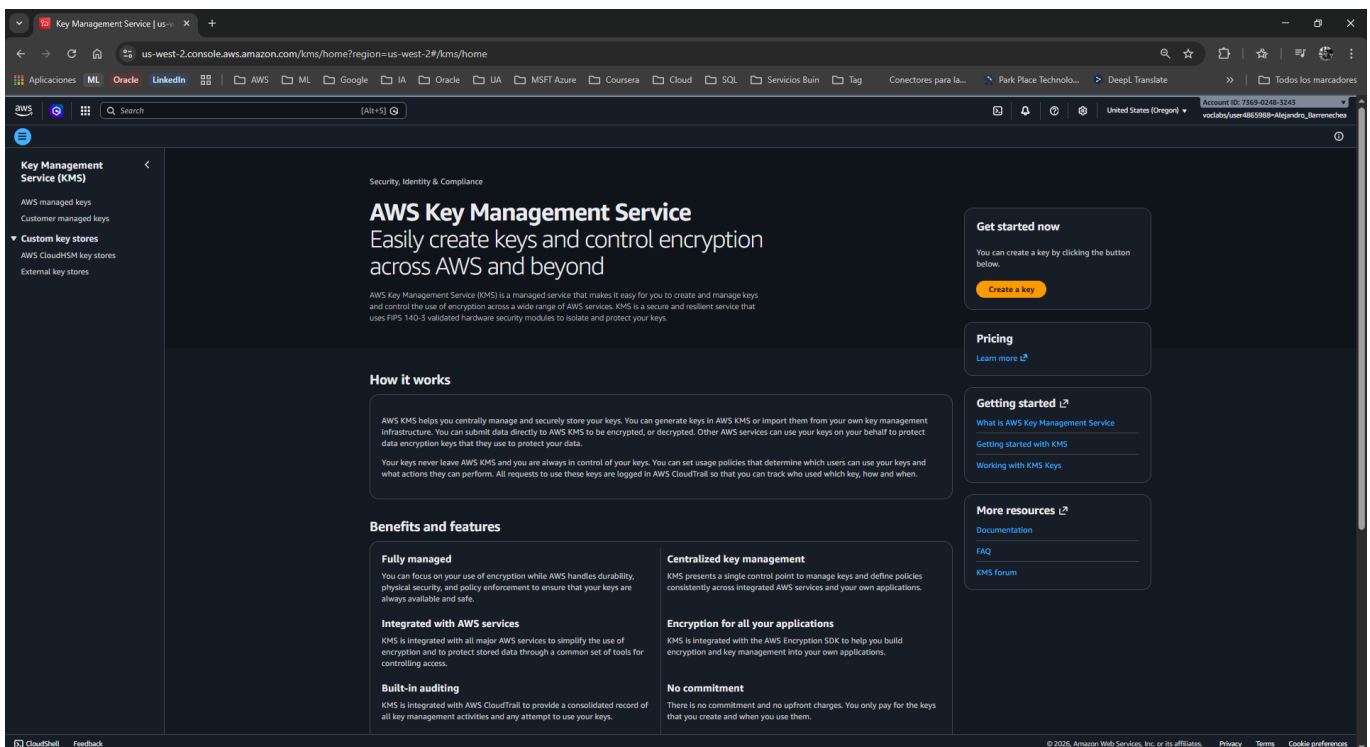
4.  **AWS IAM:** Rol y credenciales temporales (**voc1abs**) que autorizan al servidor a utilizar la clave de KMS.

Desarrollo de las Tareas (Paso a Paso)

Tarea 1: Crear una clave en AWS KMS

En esta tarea, generarás la clave maestra que protegerá tus datos.

1.  En la barra de búsqueda de la consola, escribe **KMS** y selecciona **Key Management Service**.
2.  En el panel derecho, haz clic en el botón naranja **Create a key** (Crear una clave).



3.  En **Key type** (Tipo de clave), selecciona **Symmetric** (Simétrica) y haz clic en **Next** (Siguiente).
 - *Nota analítica:* La encriptación simétrica utiliza la misma clave tanto para cifrar como para descifrar. Es extremadamente rápida e ideal para grandes volúmenes de datos.

Step 1 | Create key | KMS Console

us-west-2.console.aws.amazon.com/kms/home?region=us-west-2#/kms/keys/create

Step 1: Configure key

Step 2: Add labels

Step 3 - optional: Define key administrative permissions

Step 4 - optional: Define key usage permissions

Step 5 - optional: Edit key policy

Step 6: Review

Configure key

Key type [Help me choose](#)

☒ **Symmetric**
A single key used for encrypting and decrypting data or generating and verifying HMAC codes.

☐ **Asymmetric**
A public and private key pair used for encrypting and decrypting data, signing and verifying messages, or deriving shared secrets.

Key usage [Help me choose](#)

☒ **Encrypt and decrypt**
Use the key only to encrypt and decrypt data.

☐ **Generate and verify MAC**
Use the key only to generate and verify hash-based message authentication codes (HMAC).

Advanced options

Key material origin
Key material origin is a KMS key property that represents the source of the key material when creating the KMS key. [Help me choose](#)

☒ **KMS - recommended**
AWS KMS creates and manages the key material for the KMS key.

☐ **External (import key material)**
You create and import the key material for the KMS key.

☐ **AWS CloudHSM key store**
AWS KMS creates the key material in the AWS CloudHSM cluster of your AWS CloudHSM key store.

☐ **External key store**
The key material for the KMS key is in an external key manager outside of AWS.

Regionality
Create your KMS key in a single AWS Region (default) or create a KMS key that you can replicate into multiple AWS Regions. [Help me choose](#)

☒ **Single-Region key**
Allow this key to be replicated into other Regions.

☐ **Multi-Region key**
Allow this key to be replicated into other Regions.

Cancel Next

4. En la página **Add labels** (Agregar etiquetas), configura lo siguiente:
 - **Alias:** `MyKMSKey`
 - **Description:** `Key used to encrypt and decrypt data files.`
5. Haz clic en **Next**.

Step 2 | Create key | KMS Console

us-west-2.console.aws.amazon.com/kms/home?region=us-west-2#/kms/keys/create

Step 1: Configure key

Step 2: Add labels

Step 3 - optional: Define key administrative permissions

Step 4 - optional: Define key usage permissions

Step 5 - optional: Edit key policy

Step 6: Review

Add labels

Alias
You can change the alias at any time. [Learn more](#)

Alias

MyKMSKey

Description - optional
You can change the description at any time.

Description

Key used to encrypt and decrypt data files

Tags - optional
You can use tags to categorize and identify your KMS keys and help you track your AWS costs. When you add tags to AWS resources, AWS generates a cost allocation report for each tag. [Learn more](#)

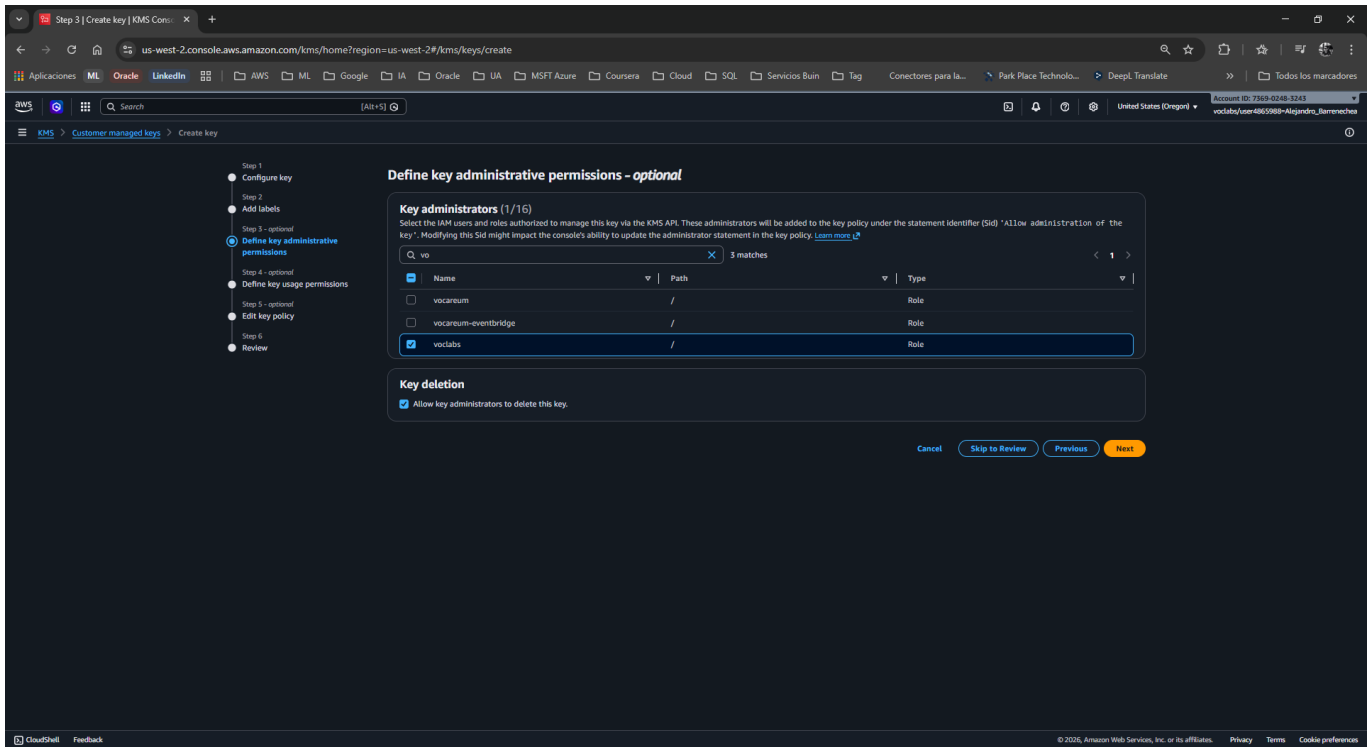
This key has no tags.

Add tag

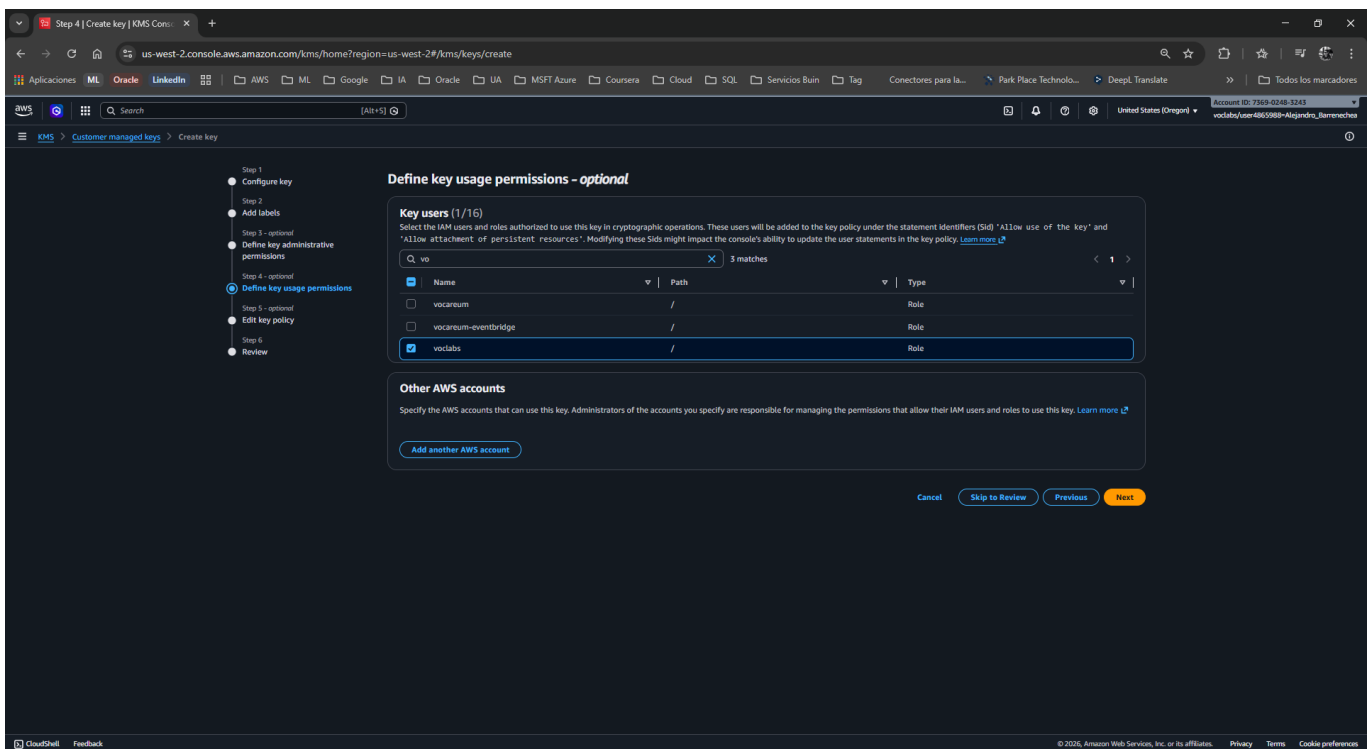
You can add up to 50 more tags.

Cancel Skip to Review Previous Next

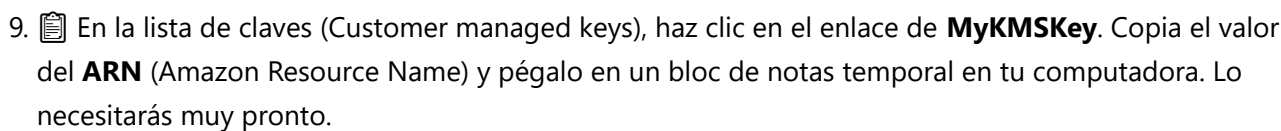
6. En **Define key administrative permissions** (Administradores de la clave), usa el buscador, marca la casilla del rol `voc1abs` y haz clic en **Next**.

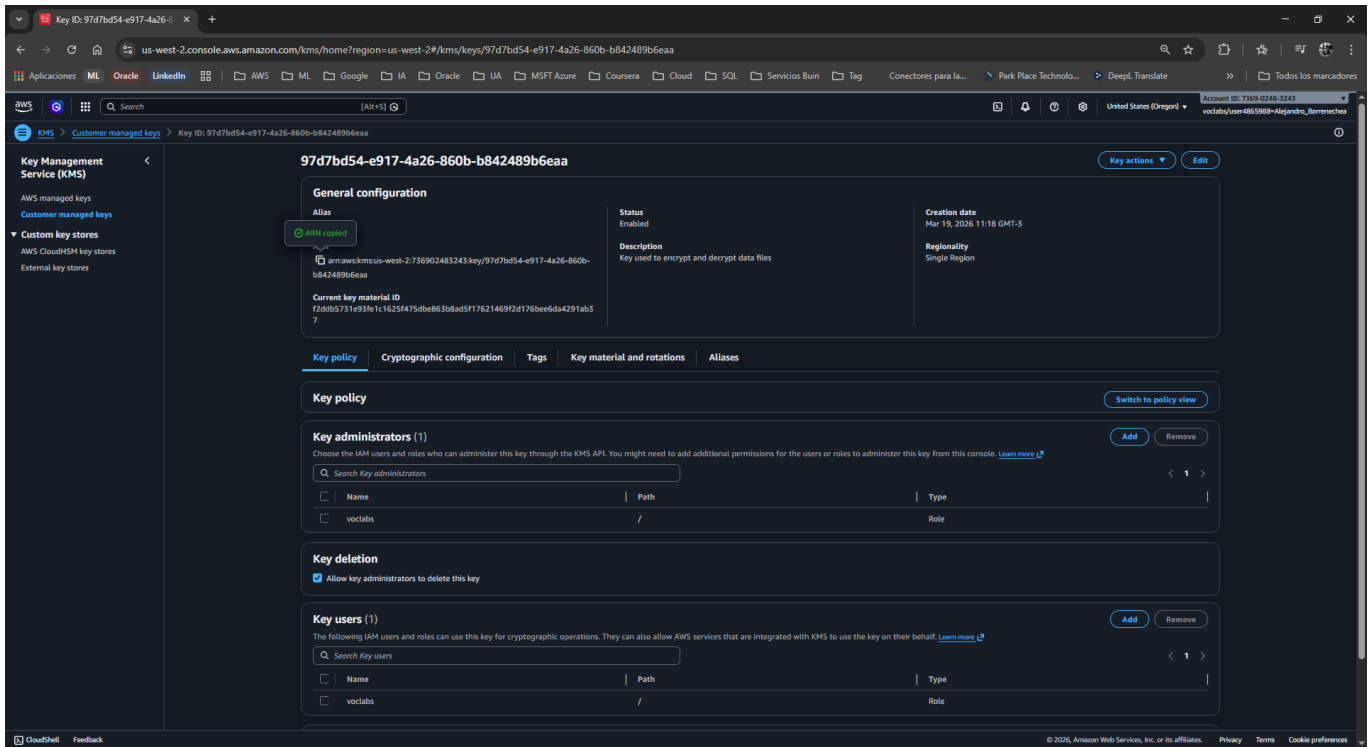


7. 👤 En **Define key usage permissions** (Usuarios de la clave), marca nuevamente la casilla del rol **voclabs** y haz clic en **Next**.



8. 👁 Revisa la configuración (las políticas JSON generadas automáticamente) y haz clic en **Finish** (Finalizar).





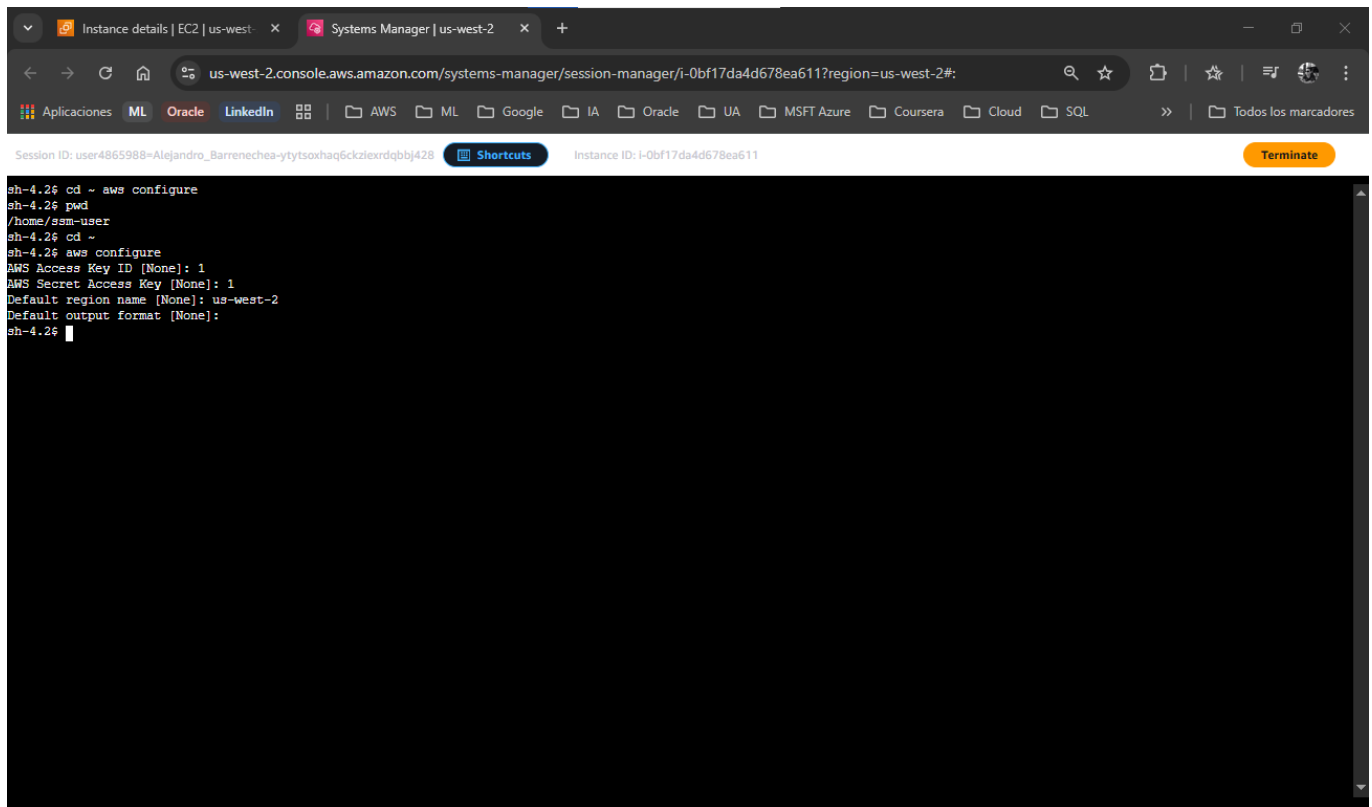
⚙️ Tarea 2: Configurar la instancia del Servidor de Archivos (File Server)

Ahora prepararás el servidor EC2 instalando las herramientas criptográficas y configurando las credenciales.

1. 🔍 En la barra de búsqueda superior, escribe **EC2** y ábrelo.
2. 📁 En el panel izquierdo, selecciona **Instances**.
3. ☒ Selecciona la instancia llamada **File Server** y haz clic en el botón superior **Connect** (Conectar).
4. 📄 Selecciona la pestaña **Session Manager** y haz clic en **Connect**. Se abrirá una terminal negra de comandos en tu navegador.
5. 💻 Ejecuta los siguientes comandos para ir al directorio de inicio e inicializar la estructura de configuración de AWS:

```
cd ~
aws configure
```

6. 🖨️ Llena los datos solicitados de la siguiente manera (presiona **Enter** después de cada uno):
 - **AWS Access Key ID:** Escribe **1**
 - **AWS Secret Access Key:** Escribe **1**
 - **Default region name:** Copia y pega la región que aparece en tu panel de Vocareum (ej. **us-east-2**).
 - **Default output format:** Presiona **Enter** (déjalo en blanco).
 - **Nota:** Los valores "1" son marcadores temporales para que el sistema cree el archivo **~/aws/credentials**.

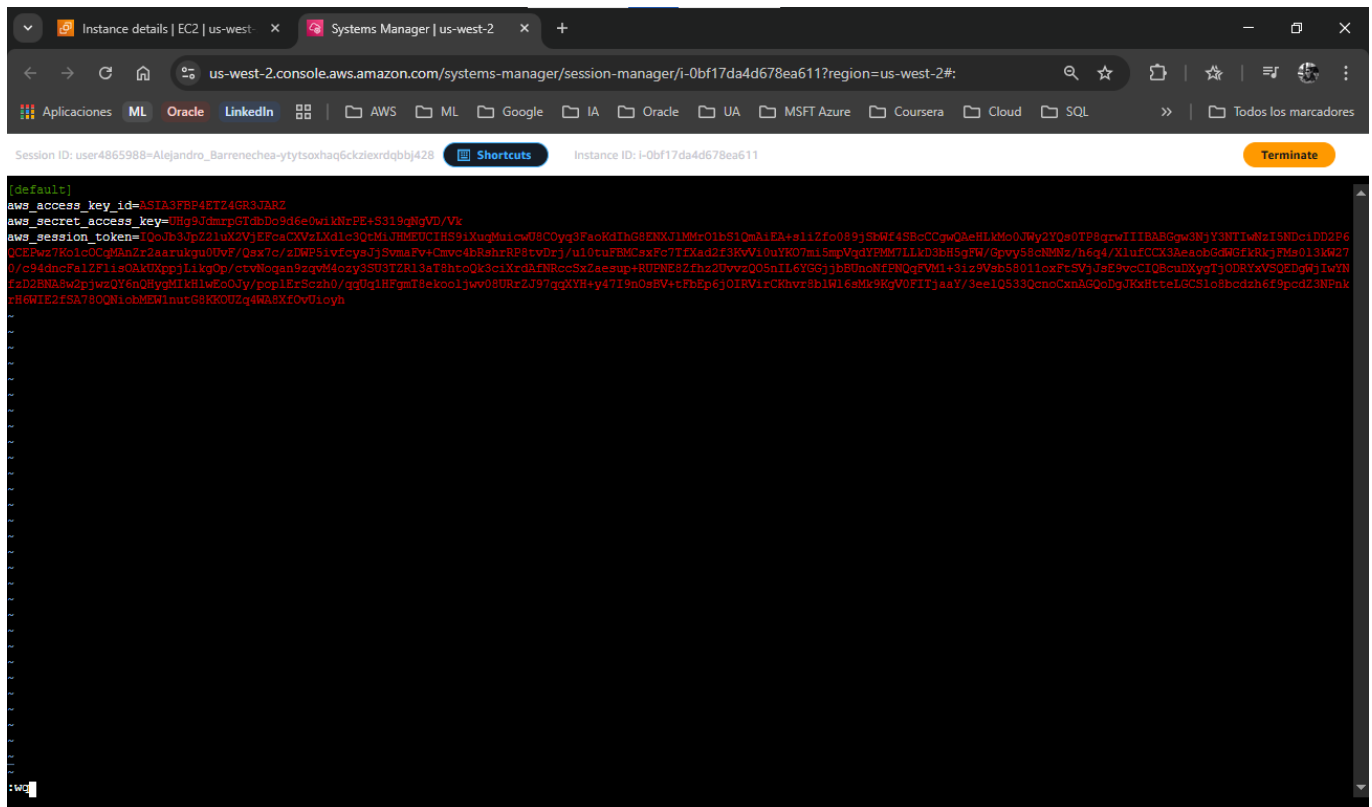


```
sh-4.2$ cd ~ & aws configure
sh-4.2$ pwd
/home/ssm-user
sh-4.2$ cd ~
sh-4.2$ aws configure
AWS Access Key ID [None]: 1
AWS Secret Access Key [None]: 1
Default region name [None]: us-west-2
Default output format [None]:
sh-4.2$
```

7.  Regresa a la pestaña de tu plataforma de laboratorio (Vocareum), haz clic en el botón **AWS Details** (arriba de *Start Lab*) y junto a **AWS CLI**, haz clic en **Show**. Copia todo el bloque de texto (incluyendo `[default]`).
8.  Vuelve a la terminal de Session Manager. Abre el archivo de credenciales con el editor `vi`:

```
vi ~/.aws/credentials
```

9.  Presiona la letra `d` dos veces (`dd`) repetidamente para borrar todas las líneas existentes.
10.  Presiona la tecla `i` para entrar en modo *Insert* (Insertar).
11.  Pega el bloque de credenciales que copiaste de Vocareum (usa `Ctrl+Shift+V` o clic derecho > Pegar).
12.  Presiona la tecla `Escape`, luego escribe `:wq` y presiona `Enter` para guardar y salir.



13. Verifica que el archivo se guardó correctamente:

```
cat ~/.aws/credentials
```

14. Instala el **AWS Encryption CLI** y configura la variable de entorno PATH ejecutando:

```
pip3 install aws-encryption-sdk-cli
export PATH=$PATH:/home/ssm-user/.local/bin
```



```

Download boto3-1.33.13-py3-none-any.whl (139 kB)
Collecting wrapt>=1.10.11
  Downloading wrapt-1.16.0-cp37-cp37m-manylinux_2_5_x86_64.manylinux1_x86_64.manylinux_2_17_x86_64.manylinux2014_x86_64.whl (77 kB)
Collecting zipp>=0.5
  Downloading zipp-3.15.0-py3-none-any.whl (6.8 kB)
Collecting typing-extensions>=3.6.4; python_version < "3.8"
  Downloading typing_extensions-4.7.1-py3-none-any.whl (33 kB)
Collecting cffi>=1.14; platform_python_implementation != "PyPy"
  Downloading cffi-1.15.1-cp37-cp37m-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (427 kB)
Collecting s3transfer<0.9.0,>=0.8.2
  Downloading s3transfer-0.8.2-py3-none-any.whl (82 kB)
Collecting botocore<1.34.0,>=1.33.13
  Downloading botocore-1.33.13-py3-none-any.whl (11.8 MB)
Collecting jmespath<2.0.0,>=0.7.1
  Downloading jmespath-1.0.1-py3-none-any.whl (20 kB)
Collecting pycparser
  Downloading pycparser-2.21-py2.py3-none-any.whl (118 kB)
Collecting python-dateutil<3.0.0,>=2.1
  Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)
Collecting urllib3<1.27,>=1.25.4; python_version < "3.10"
  Downloading urllib3-1.26.20-py2.py3-none-any.whl (144 kB)
Collecting six>=1.5
  Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)
Installing collected packages: zipp, typing-extensions, importlib-metadata, attrs, base64io, pycparser, cffi, cryptography, six, python-dateutil, urllib3, jmespath, botocore, s3transfer, boto3, wrapt, aws-encryption-sdk, aws-encryption-sdk-cli
WARNING: The script aws-encryption-cli is installed in '/home/ssm-user/.local/bin' which is not on PATH.
Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed attrs-24.2.0 aws-encryption-sdk-3.3.0 aws-encryption-sdk-cli-4.3.0 base64io-1.0.3 boto3-1.33.13 botocore-1.33.13 cffi-1.15.1 cryptography-45.0.7 importlib-metadata-6.7.0 jmespath-1.0.1 pycparser-2.21 python-dateutil-2.9.0.post0 s3transfer-0.8.2 six-1.17.0 typing-extensions-4.7.1 urllib3-1.26.20 wrapt-1.16.0 zipp-3.15.0
sh-4.2$ export PATH=$PATH:/home/ssm-user/.local/bin
sh-4.2$

```

Tarea 3: Encriptar y descifrar datos

En esta fase final, crearás datos sensibles y probarás el motor criptográfico local.

1. Crea tres archivos de texto y añade contenido secreto al primero:

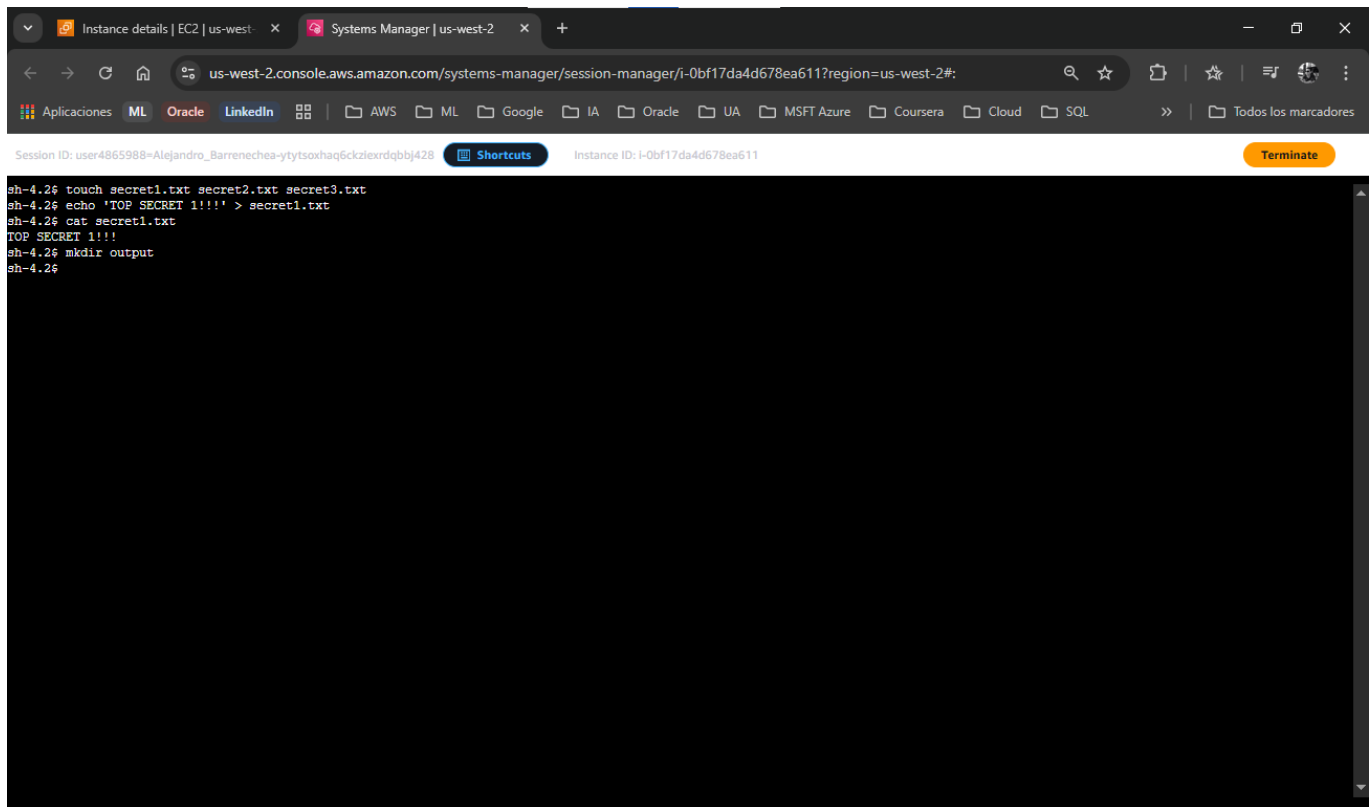
```
touch secret1.txt secret2.txt secret3.txt
echo 'TOP SECRET 1!!!!' > secret1.txt
```

2. Verifica el contenido en texto plano legible:

```
cat secret1.txt
```

3. Crea un directorio para guardar los resultados cifrados:

```
mkdir output
```



4. 🔑 Define una variable de entorno con el ARN de tu clave KMS (reemplaza **(KMS ARN)** con el ARN exacto que guardaste en tu bloc de notas en la Tarea 1):

```
keyArn=(KMS ARN)
```

5. 📁 **¡Encripta el archivo!** Ejecuta el siguiente comando bloque (cópialo y pégalo completo):

```
aws-encryption-cli --encrypt \
  --input secret1.txt \
  --wrapping-keys key=$keyArn \
  --metadata-output ~/metadata \
  --encryption-context purpose=test \
  --commitment-policy require-encrypt-require-decrypt \
  --output ~/output/.
```

6. 🖨️ Valida si el comando fue exitoso (un resultado de 0 significa éxito):

```
echo $?
```

```

sh-4.2$ aws-encryption-cli --encrypt \
> --input secret1.txt \
> --wrapping-keys key=$keyArn \
> --metadata-output ~/metadata \
> --encryption-context purpose=test \
> --commitment-policy require-encrypt-require-decrypt \
> --output ~/output/
/home/ssm-user/.local/lib/python3.7/site-packages/aws_encryption_sdk/caches/_init_.py:11: CryptographyDeprecationWarning: Python 3.7 is no longer supported by the Python core team a
nd support for it is deprecated in cryptography. The next release of cryptography will remove support for Python 3.7.
  from cryptography.hazmat.backends import default_backend
Traceback (most recent call last):
  File "/home/ssm-user/.local/bin/aws-encryption-cli", line 5, in <module>
    from aws_encryption_sdk_cli import cli
  File "/home/ssm-user/.local/lib/python3.7/site-packages/aws_encryption_sdk_cli/_init_.py", line 31, in <module>
    from aws_encryption_sdk_cli.internal.master_key_parsing import build_crypto_materials_manager_from_args
  File "/home/ssm-user/.local/lib/python3.7/site-packages/aws_encryption_sdk_cli/internal/master_key_parsing.py", line 17, in <module>
    from importlib.metadata import EntryPoint, distributions
ModuleNotFoundError: No module named 'importlib.metadata'
sh-4.2$ echo $?
1
sh-4.2$ ls output
sh-4.2$ ls
output  secret1.txt  secret2.txt  secret3.txt
sh-4.2$ cd output
sh-4.2$ cat secret1.txt.encrypted
cat: secret1.txt.encrypted: No such file or directory
sh-4.2$ pwd
/home/ssm-user/output
sh-4.2$ cat secret1.txt.encrypted
cat: secret1.txt.encrypted: No such file or directory
sh-4.2$ ls
sh-4.2$ cd..
sh: cd.: command not found
sh-4.2$ cd ..
sh-4.2$ ls
output  secret1.txt  secret2.txt  secret3.txt
sh-4.2$ cat secret1.txt.encrypted
cat: secret1.txt.encrypted: No such file or directory
sh-4.2$

```

Durante la ejecución del laboratorio de protección de datos (Tarea 3, Paso 5), el comando de encriptación falló, arrojando un código de salida **1** (`echo $?`).

Estrategia de Solución: En un entorno de producción, la solución ideal sería actualizar el sistema operativo o la versión de Python. Sin embargo, en un entorno de laboratorio efímero, la solución más rápida es instalar manualmente el *backport* (parche de compatibilidad) de esa librería específica para que Python 3.7 pueda entenderla.

1. Visualiza el archivo encriptado y trata de leerlo:

```

cd output
ls
cat secret1.txt.encrypted

```

- *Diagnóstico visual:* Observarás caracteres ilegibles y símbolos extraños. El texto original **TOP SECRET 1!!!** se ha transformado en un *Ciphertext* impenetrable.

2. **¡Descifra el archivo!** Regresa el archivo a su estado original (presiona *Enter* primero si tu terminal quedó desordenada por los caracteres extraños). Ejecuta:

```

aws-encryption-cli --decrypt \
--input secret1.txt.encrypted \
--wrapping-keys key=$keyArn \
--commitment-policy require-encrypt-require-decrypt \
--encryption-context purpose=test \
--metadata-output ~/metadata \
--max-encrypted-data-keys 1 \

```

```
--buffer \  
--output .
```

3. 👁 Verifica el resultado final listando y leyendo el nuevo archivo descifrado:

```
ls  
cat secret1.txt.encrypted.decrypted
```

- *Validación:* El texto **TOP SECRET 1!!!** volverá a aparecer perfectamente legible en tu pantalla.
[📷 Inserta tu captura aquí: (Captura de la terminal mostrando el comando de descifrado exitoso y la lectura del archivo .decrypted restaurando el texto "TOP SECRET 1!!!")]

💡 Respuestas Analíticas al Caso

¿Por qué utilizamos Encriptación Simétrica y no Asimétrica para los archivos?

- **Respuesta:** La criptografía simétrica (donde la misma clave cifra y descifra) utiliza algoritmos mucho más rápidos y menos intensivos en CPU (como AES-256). Es el estándar de la industria para cifrar grandes volúmenes de datos (archivos, bases de datos). La criptografía asimétrica (llave pública/privada) es matemáticamente lenta y se usa típicamente para establecer canales seguros (TLS/SSL) o firmar digitalmente, no para cifrar *Data at Rest* de forma masiva.

¿Qué función cumple exactamente el AWS Encryption CLI y qué es el Cifrado de Sobres?

- **Respuesta:** Enviar Gigabytes de archivos a través de Internet hacia AWS KMS para ser cifrados sería ineficiente y costoso. El *AWS Encryption CLI* soluciona esto mediante el **Cifrado de Sobres (Envelope Encryption)**. En este modelo, el CLI le pide a KMS una "Clave de Datos" (*Data Key*). El CLI usa esa clave de datos en la memoria RAM del servidor local para cifrar el archivo a máxima velocidad. Luego, guarda el archivo cifrado y le "pega" como metadato la clave de datos (la cual viene envuelta/cifrada por la clave maestra de KMS).